Hotel Bookings — Answers (Sections 1–3)

Dataset: `hotel_bookings.csv`, 12,000 bookings, 800 customers, 60 properties. All numbers below are produced by the scripts in `code/` (`analysis.py`, `run_sql.py`) run against the footnote-aware cleaning in `code/clean.py`; nothing here is hand-typed. Where a footnote matters I say so inline.

Cleaning baseline (all footnotes handled in `clean.py`): `keep_default_na=False` so the loyalty tier `'None'` survives (Footnote 7 — otherwise 5,571 rows / 46% go missing). Flagged, not silently dropped: 120 invalid stays (F1), 163 bookings before signup (F2), 60 zero-room rows (F3), 50 reviews on cancelled bookings (F5). Structural errors (F1, F3) are excluded from analysis → 11,821 analysable rows. Realized revenue = `Completed` only (F8).

Section 1 — Data Quality Checks

B1. Property names appearing in >1 city (different `property_id`) — 4 of them:

property_name	cities (property_id)
The Grand Plaza	Bangalore (2), Kochi (1)
Sea Breeze Resort	Chennai (3), Goa (4)
Hilltop Inn	Chennai (5), Udaipur (6)
Royal Orchid	Chennai (7), Goa (8)

This is Footnote 4 in the flesh — grouping on `property_name` would silently merge two different hotels. Always group on `property_id`.

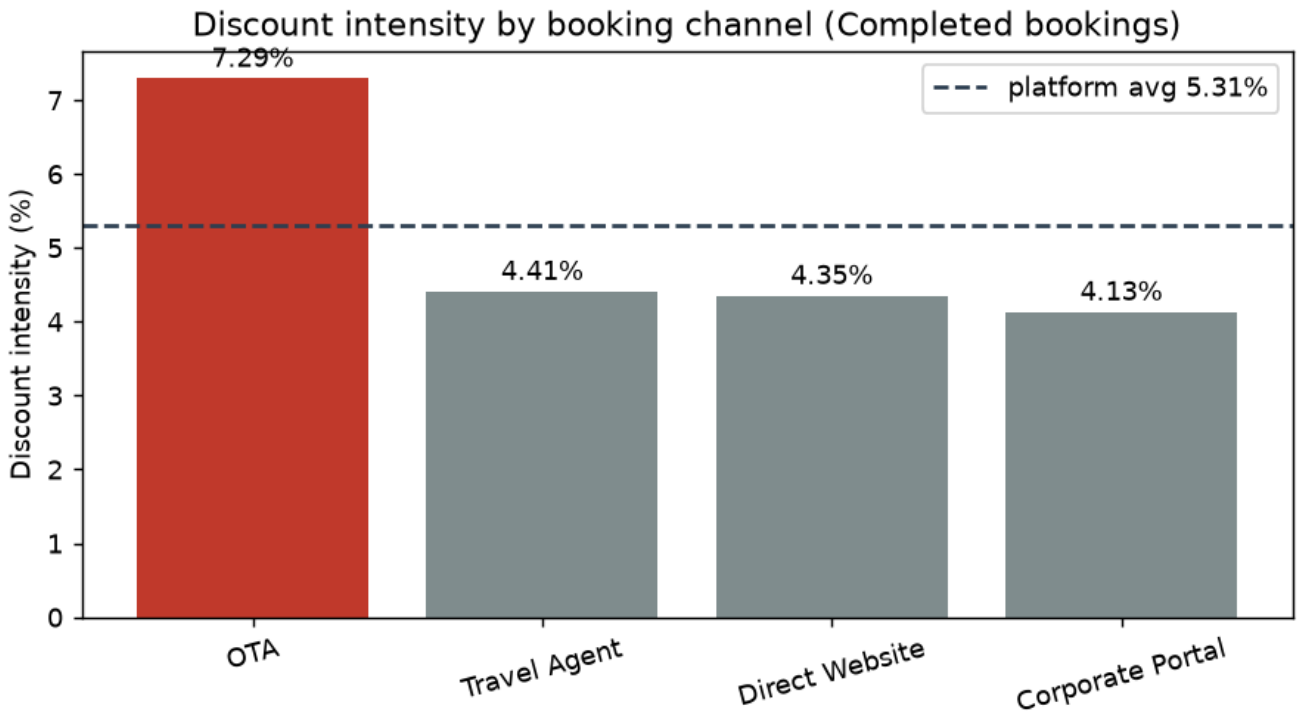
B2. Two encodings for "no coupon": the literal string `'NONE'` (3,978 rows) and the **empty string** `''` (3,894 rows). Everything else is a real coupon → **4,128 / 12,000 = 34.4%** of bookings use a real coupon.

B3. Review ratings on Cancelled bookings: 50. Impossible because a review can only follow a completed stay — a cancelled booking was never fulfilled, so there is nothing to rate (Footnote 5). **Rule:** treat the review on any `Cancelled` (or `No-Show`) booking as invalid — null out `review_rating`/`review_date` and exclude it from all rating aggregates (that is exactly what `run_sql.py` does before loading `reviews`).

Section 2 — Case Study: The Discount Erosion Problem

B1 — Discount landscape (visualization)

Discount intensity = Σ`discount_amount` ÷ Σ`gross` (gross = `total_amount` + `discount_amount`), both over **Completed** bookings.



channel	intensity	gap vs platform
OTA	**7.29%**	**+1.98 pp**
Travel Agent	4.41%	–0.91 pp
Direct Website	4.35%	–0.96 pp
Corporate Portal	4.13%	–1.18 pp

Platform-wide intensity = 5.31%. The discount spend is concentrated on **OTA**, which sits **+1.98 pp** above the platform average — the next three channels are all *below* average and clustered near 4.1–4.4%. OTA is the leak. It is the focus channel for the rest of the section.

B2 — Is OTA different because of WHO uses it, or the channel itself?

Mix tables, OTA vs platform (analysable rows; tier reading respects Footnote 7):

By segment (%)

segment	OTA	platform
Individual	72.1	64.1
Corporate	14.1	24.2
Group	13.8	11.8

By loyalty tier (%)

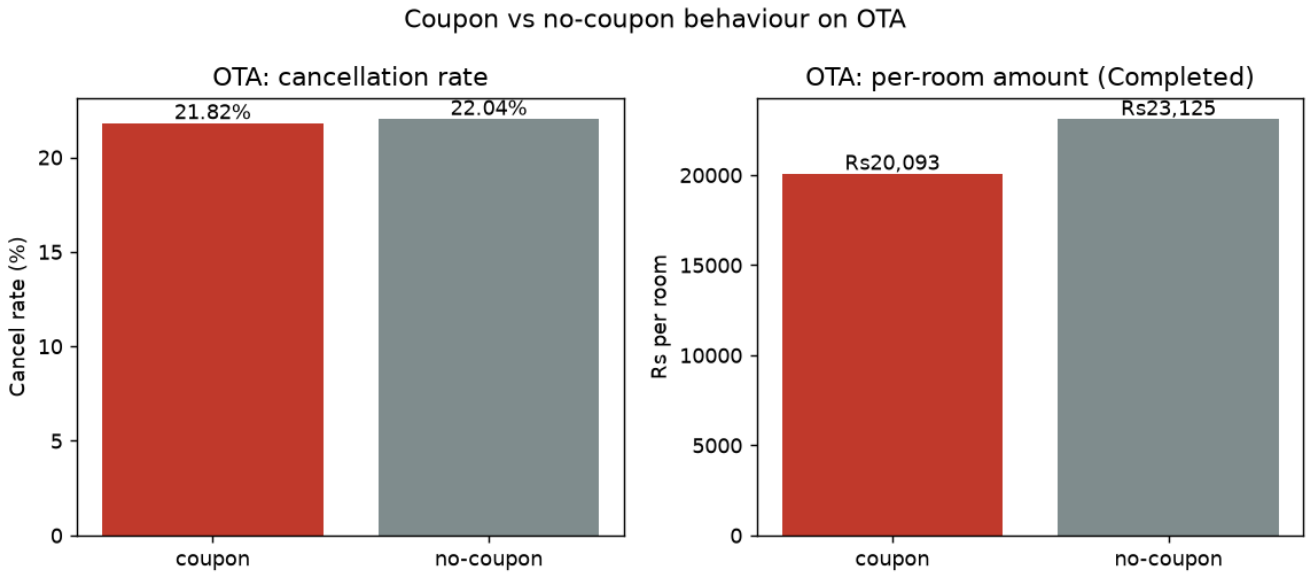
tier	OTA	platform
None	44.1	46.4
Silver	26.8	26.1
Gold	19.0	17.5
Platinum	10.1	9.9

Source-city pattern: booked outside home city — OTA 89.5% vs platform 89.8% (no meaningful difference).

Read: OTA skews a bit more Individual (72% vs 64%) and less Corporate, but its **loyalty mix is almost identical to the platform** (tier shares within ~2 pp) and travel pattern is identical. A modest tilt toward Individual customers cannot explain a discount intensity **76% higher** than every other channel. The heavy discounting is a property of **the channel's own pricing behaviour**, not of a fundamentally different customer base — i.e. it's a lever we control, not a customer we're forced to subsidise.

B3 — Are the coupons doing anything? (visualization)

Within **OTA only**, coupon vs no-coupon:



metric	coupon (n=1,416)	no-coupon (n=2,768)
cancellation rate	**21.82%**	22.04%
per-room amount (Completed)	**₹20,093**	₹23,125

(Repeat-behaviour was requested in the briefing but can't be isolated here: with only 800 customers across 12,000 bookings every customer is a "repeat" by construction, so the metric is degenerate — hence the two discriminating metrics above.)

Verdict — coupons attract identical-or-worse behaviour. Cancellation is flat (21.82% vs 22.04%, a 0.2 pp difference — noise), so coupons buy **no** commitment lift. And coupon bookings carry a **13% lower per-room basket** (₹20,093 vs ₹23,125) — we discount *and* the customer spends less per room. That is textbook margin leakage: money handed back with no behavioural return.

B4 — Recommendation + one leading indicator

Pullback strategy: Discontinue OTA coupon codes for repeat/known customers and cap the remaining OTA promo to a single acquisition coupon. Concretely, retire the three worst- value codes (`APPDEAL`, `FEST15`, `SAVE10`) on OTA and keep only `WELCOME` for genuinely new accounts. The four OTA codes each cost ₹1.7–2.2 M on completed bookings.

Expected margin recovery: realized OTA coupon discount on Completed bookings = **₹7.72 M** (7.86% of OTA's ₹98.2 M realized revenue). Retiring the three codes and keeping `WELCOME` recovers ~**₹6.06 M/period** (₹2.17 M + ₹2.08 M + ₹1.81 M). Since cancellation is statistically identical with and without coupons, the volume genuinely at risk is minimal, so most of that ₹6 M is recoverable margin rather than forgone revenue.

Leading indicator to monitor (first 60 days): OTA weekly Completed-booking volume (count of `booking_status='Completed'` on OTA per ISO week). It's the cleanest early read on whether pulling coupons is suppressing demand. **Abort threshold:** if OTA weekly completed volume falls **>8%** below the pre-pullback 4-week baseline for **two consecutive weeks**, halt the rollback — an 8% volume loss roughly offsets the ~7.9% discount saving, so beyond that we're trading margin for nothing.

Section 3 — SQL Challenge

Full DDL is in `code/schema.sql`; queries in `code/queries.sql`; both are executed end-to-end by `code/run_sql.py` (loads the cleaned CSV into SQLite using the normalized schema and runs the queries — output pasted below is real).

Schema design (normalized, 4 tables)

Denormalized customer/property attributes are split into their own tables keyed by id. Money → `DECIMAL(p,s)`, free text → bounded `VARCHAR`, dates → `DATE`. Highlights:

- **customers** — PK `customer_id`; `loyalty_tier VARCHAR(10) NOT NULL CHECK (tier IN ('None','Silver','Gold','Platinum'))` — encodes Footnote 7 (`'None'` is a value, not NULL).
- **properties** — PK `property_id`; **`UNIQUE(property_name, property_city)`** — the Footnote 4 constraint: a name may repeat across cities but not within one, and it documents that name alone is never a key.
- **bookings** — PK `booking_id`, FKs to customers/properties; money as `DECIMAL(12,2)`; **`CHECK(checkout_date > checkin_date)`** (Footnote 1) and **`CHECK(num_rooms > 0)`** (Footnote 3); `booking_status` domain-checked.
- **reviews** — PK/FK `booking_id`, `review_rating DECIMAL(3,1)` (raw scale; Footnote 6 normalization is applied in analysis, not stored).

Index chosen: `CREATE INDEX ix_bookings_status_prop_checkin ON bookings(booking_status, property_id, checkin_date);` — *both* queries start by filtering `booking_status='Completed'` then either join to `properties` (B-Q1) or bucket by `checkin_date` month (B-Q2). A composite index led by `booking_status` lets the engine seek the Completed slice and have the join key and date already ordered, instead of full-scanning `bookings`.

B-Q1 — Top room type per property type (`DENSE_RANK`)

```
WITH counts AS (
   SELECT p.property_type, b.room_type, COUNT(*) AS booking_count
   FROM bookings b
   JOIN properties p ON p.property_id = b.property_id
   WHERE b.booking_status = 'Completed'
   GROUP BY p.property_type, b.room_type
),
ranked AS (
   SELECT property_type, room_type, booking_count,
          DENSE_RANK() OVER (PARTITION BY property_type
                             ORDER BY booking_count DESC) AS rnk
   FROM counts
)
SELECT property_type, room_type, booking_count
FROM ranked WHERE rnk = 1
ORDER BY property_type;
```

property_type	room_type	booking_count
Budget	Standard	1,456
Luxury	Standard	531
Mid-Range	Standard	1,767
Premium	Standard	808

Why `DENSE_RANK`: `ROW_NUMBER` would pick one arbitrary winner and silently drop a genuine tie; `RANK` returns all tied winners but leaves gaps in the ranking. `DENSE_RANK` keeps **every** tied top room type at rank 1 with no gap — exactly "the most-booked room type(s) per type."

B-Q2 — Monthly realized revenue 2024 + running cumulative

```
WITH monthly AS (
   SELECT strftime('%Y-%m', checkin_date) AS month,
          SUM(total_amount) AS monthly_revenue
   FROM bookings
   WHERE booking_status = 'Completed'
     AND strftime('%Y', checkin_date) = '2024'
   GROUP BY strftime('%Y-%m', checkin_date)
)
SELECT month, monthly_revenue,
      SUM(monthly_revenue) OVER (ORDER BY month
          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_revenue
FROM monthly ORDER BY month;
```

month	monthly_revenue	cumulative_revenue
2024-01	27,769,879.35	27,769,879.35

2024-02	23,224,199.67	50,994,079.02
2024-03	21,772,021.83	72,766,100.85
2024-04	22,842,492.34	95,608,593.19
2024-05	24,184,175.95	119,792,769.14
2024-06	21,486,001.48	141,278,770.62
2024-07	19,525,948.06	160,804,718.68
2024-08	22,494,670.72	183,299,389.40
2024-09	24,266,242.91	207,565,632.31
2024-10	26,944,719.99	234,510,352.30
2024-11	28,144,993.11	262,655,345.41
2024-12	29,540,760.82	292,196,106.23

Full-year cumulative realized revenue = ■292,196,106.23 (Completed only — Footnote 8; summing all statuses would overstate this by ~30%).

Section 4 — Mini-Project

See `project/` — `holiday_tagger.py`, `README.md`, and the generated `holiday_value.png`. Headline: holiday-adjacent bookings (±2 days of a 2024 India public holiday, 20.3% of bookings) are worth **4.2% less** (■30,704 vs ■32,033), stay the same length (2.87 vs 2.85 nights), and cancel **1.71 pp more** (20.5% vs 18.8%) — i.e. no long-weekend premium here.